

Structure générale d'un script PHP

Balises PHP

- Lorsque PHP traite un fichier, il cherche les balises d'ouverture et de fermeture (**<?php** et **?>**) qui délimitent le code qu'il doit interpréter.
- En version 7, PHP accepte deux syntaxes pour les balises:
 - `<?php... ?>`
 - `<? ... ?>`
- Avant la version 7, PHP acceptait deux syntaxes supplémentaires pour les balises:
 - `<script language="php"> ... </script>`
 - `<% ... %>`

1

Structure générale d'un script PHP

Balises PHP

- Tout ce qui est à l'extérieur de la balise PHP est transmis tel quel au navigateur; seul les instructions PHP sont interprétées par le moteur PHP.

Information

<code>//</code>	: permet d'ajouter un commentaire sur une seule ligne
<code>#</code>	: permet d'ajouter un commentaire sur une seule ligne
<code>/* ... */</code>	: permet d'ajouter un commentaire sur plusieurs lignes

2

Structure générale d'un script PHP

Balises PHP

```
<?php
// Commentaire sur une seule ligne, style C++
/*
Ceci est un
commentaire
sur
Plusieurs lignes
*/
# Ceci est un commentaire style shell sur une seule ligne
?>
```

3

Structure générale d'un script PHP

La fonction echo

- echo Affiche une chaîne de caractères
- PHP inclut une balise ouvrante echo courte **<?=** qui est un raccourci au code plus verbeux **<?php echo**



Information

- PHP requiert que les instructions soient terminées par un point-virgule à la fin de chaque instruction.

4

Structure générale d'un script PHP

La fonction echo

```
<?php
echo 'Ceci est un test';
echo 'Ceci est un autre test';
?>

<?=
'Et un test final';
?>
```

5

Manipulation des Variables/constantes/Affectation

Les variables essentielles

En PHP, les variables sont représentées par un signe dollar "\$ » suivi du nom de la variable.

Un nom de variable valide doit respecter les règles suivantes (après le symbole \$):

- Le nom est sensible à la casse : \$a et \$A sont deux variables distinctes.
- Le premier caractère doit être une lettre ou un souligné _
- À partir du deuxième caractère seul les lettres, chiffres ou soulignés sont acceptés.

6

Manipulation des Variables/constantes/Affectation

Les variables essentielles

```
<? php
$var;    //nom de variable valide
$Var;    //nom de variable valide
$4vars;  //nom de variable invalide: commencer avec un nombre n'est pas autorisé
$_var;   //nom de variable valide
$état;   //nom de variable valide
$éta+;   //nom de variable invalide: les caractères spéciaux comme + ne sont pas autorisés
$var Vat; //nom de variable invalide: l'espace n'est pas autorisé
?>
```

7

Manipulation des Variables/constantes/Affectation

Variables prédéfinies

Les variables "**superglobales** » sont disponibles quel que soit le contexte du script.

\$GLOBALS Référence toutes les variables disponibles dans un contexte global		
\$_SERVER Variables de serveur et d'exécution	\$_GET Variables HTTP GET	\$_POST Variables HTTP POST
\$_FILES Variable de téléchargement de fichier via HTTP	\$_REQUEST Variables de requête HTTP	\$_SESSION Variables de session
\$_ENV Variables d'environnement	\$_COOKIE Cookies HTTP	\$php_errormsg Le dernier message d'erreur
\$http_response_header En-têtes de réponse HTTP	\$argc Le nombre d'arguments passés au script	\$argv Tableau d'arguments passés au script

8

Manipulation des Variables/constantes/Affectation

Variables prédéfinies

Les variables "**superglobales** » sont disponibles quel que soit le contexte du script.

```
<? php
echo 'Adresse IP du serveur web : '.$_SERVER['SERVER_ADDR'];
echo 'Nom du serveur web : '. $_SERVER['HTTP_HOST'] ;
echo 'Adresse IP du client : '. $_SERVER['REMOTE_ADDR'];
?>
```

Manipulation des Variables/constantes/Affectation

Portée des variables

- Une variable globale doit être déclarée à l'intérieur de chaque fonction afin de pouvoir être utilisée dans cette fonction.
- Il suffit d'utiliser le mot clé **globals** avant la variable pour l'utiliser.

Partie 2 - Chapitre 1 : Maitriser le langage PHP

Manipulation des Variables/constantes/Affectation

Portée des variables

```
<?php

$var = "Exemple de contenu";

function test() {
    $var = "variable locale";
    echo '$var dans le contexte global : ' . $GLOBALS["var"] ;
    echo '$var dans le contexte courant : ' . $var ;
}
test();

?>
```

11

Manipulation des Variables/constantes/Affectation

Exemples fonctions de gestion des variables

get_defined_vars

Liste toutes les variables définies

floatval

Convertit une chaîne en nombre à virgule flottante

get_resource_type

Retourne le type de ressource

is_array

Détermine si une variable est un tableau

empty

Détermine si une variable est vide

is_object

Détermine si une variable est de type objet

isset

Détermine si une variable est déclarée et est différente de null

get_resource_id

Retourne un entier identifiant une ressource

unset

Détruit une variable

var_dump

Affiche les informations d'une variable

is_null

Indique si une variable vaut null

print_r

Affiche des informations lisibles pour une variable

var_export

Retourne le code PHP utilisé pour générer une variable

12

Manipulation des Variables/constantes/Affectation

Les constantes

- Une constante est un identifiant (un nom) qui représente une valeur simple.
- Les constantes sont sensibles à la casse.
- Par convention, les constantes sont toujours en majuscule.
- Les constantes peuvent être définies en utilisant le mot clé **const** ou en utilisant la fonction **define()**.
- Une constante n'est pas préfixée d'un \$
- Pour vérifier qu'une constante est définie, utiliser la fonction **defined()**.
- Si une constante indéfinie est utilisée une Error est renvoyée.
- Contrairement aux constantes définies en utilisant l'instruction `define()`, les constantes définies en utilisant le mot-clé `const` doivent être déclarées au plus haut niveau du contexte, car elles seront définies au moment de la compilation.

13

Manipulation des Variables/constantes/Affectation

Les constantes

```
<? Php
const TEST= 'Bonjour les étudiants !';
echo TEST; //Affiche Bonjour les étudiants !
?>
```

14

Manipulation des Variables/constantes/Affectation

Les opérateurs d'affectation

- L'opérateur d'affectation le plus simple est le **signe =**
- Il signifie que l'opérande de gauche se voit affecter la valeur de l'expression qui est à droite du signe égal.

15

Manipulation des Variables/constantes/Affectation

Les opérateurs d'affectation

- Exemples

```
<?php
$a = 2; //$a est maintenant égal à 2
$b = 3; //$b est maintenant égal à 3
$c = $a + $b; //$c est maintenant égal à 5
$d = $$c; //$d est maintenant égal à 5
$e = "Bonjour ";
$e .= " les étudiants"; // affecte la valeur "Bonjour les étudiants" à la variable e
$c += $a*2 // $c est maintenant égal à 9 de même que $d est maintenant égal à 9
?>
```

16

Manipulation des Variables/constantes/Affectation

Exercice1:

Parmi les variables suivantes, lesquelles ont un nom valide :

\$a, \$_a, \$a_a, \$AAA, \$a!, \$1a et \$a1 ?

17

Manipulation des Variables/constantes/Affectation

Exercice1:

Parmi les variables suivantes, lesquelles ont un nom valide :

\$a, \$_a, \$a_a, \$AAA, \$a!, \$1a et \$a1 ?

Réponse:

Seules les variables **\$a**, **\$_a**, **\$a_a**, **\$AAA** et **\$a1** ont un nom valide en PHP.

\$1a n'est pas un nom de variable valide car il commence par un chiffre, tandis que \$a! est également incorrect car il contient un caractère interdit.

18

Manipulation des Variables/constantes/Affectation

Exercice2:

Calculer le prix TTC du produit en €.

Sachant que : Prix HT=60 et TVA=20

19

Manipulation des Variables/constantes/Affectation

Exercice2:

Calculer le prix TTC du produit en €.

Sachant que : Prix HT=60 et TVA=20

Réponse:

```
<?php
$prix_ht = 60;
$tva = 20;
$prix_ttc = $prix_ht * (1 + ($tva / 100));
echo 'Le prix TTC du produit est de ' . $prix_ttc . ' €.';
?>
```

20

Manipulation des Variables/constantes/Affectation

Exercice3:

Déclarer une variable \$test qui contient la valeur 42. En utilisant une fonction de gestion des variables, faire en sorte que le type de la variable \$test soit **string** et que la valeur soit bien de **42**.

21

Manipulation des Variables/constantes/Affectation

Exercice3:

Déclarer une variable \$test qui contient la valeur 42. En utilisant une fonction de gestion des variables, faire en sorte que le type de la variable \$test soit **string** et que la valeur soit bien de **42**.

Réponse:

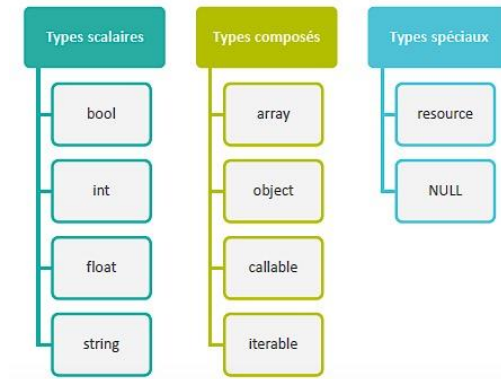
```
<?php
$test = '42';
var_dump($test);
?>
```

22

Manipulation des types de données

Introduction

- Les variables PHP pourront stocker différents types de valeurs comme des nombres ou des tableaux. Par abus de langage, on parle des « **types variables** » de PHP.
- PHP supporte 10 types basiques:



23

Manipulation des types de données

Type booléen

- Il peut avoir **true** ou **false**.
 - **==** est un opérateur qui teste l'égalité et retourne un booléen.
 - Pour convertir explicitement une valeur en booléen, utilisez (bool) ou (boolean).
- Lors d'une conversion en booléen, les valeurs suivantes sont considérées comme false:
- false
 - l'entier 0, les nombres à virgule flottante 0.0 et -0.0
 - la chaîne vide, et la chaîne "0"
 - un tableau avec aucun élément
 - le type spécial NULL (incluant les variables non définies)
 - les objets Simple XML créés depuis des éléments vide sans attributs.

24

Manipulation des types de données

Type booléen

```
<? php
$bool_val= (bool) true;
echo $bool_val; // Affiche : 1
$bool_val2 = (bool) false;
echo $bool_val2; // N'affiche rien
$bool_exp1 = (bool) false;
echo $bool_exp1 ? 'true' : 'false'; // Affiche : false
?>
```

25

Manipulation des types de données

Type entier

Un entier est un nombre appartenant à l'ensemble $\mathbb{Z}=\{\dots,-2,-1,0,1,2,\dots\}$.

Les entiers peuvent être spécifiés en notation:

- Décimale (base 10)
- Hexadécimale (base 16) : Pour utiliser la notation hexadécimale, précédez le nombre de 0x
- Octale (base 8) : Pour utiliser cette notation, précédez le nombre d'un 0 (zéro). (à partir PHP 8.1.0, la notation octale peut être précédé avec 0o ou 0O)
- Binaire (base 2) : Pour utiliser la notation binaire, précédez le nombre de 0b.
- L'opérateur de négation peut être utilisé pour désigner un entier négatif.

26

Manipulation des types de données

Type entier

- À partir de PHP 7.4.0, les entiers littéraux peuvent contenir des underscores (_) entre les chiffres, pour une meilleure lisibilité des littéraux. Ces underscores sont supprimés par le scanner de PHP.

Exemple: \$a= 1_203_489 // c'est la même chose que \$a=1203489

- Pour convertir explicitement une valeur en un entier, utiliser soit le mot-clé (int), soit(integer).
- Depuis un booléen : false correspond à 0, et true correspond à 1.
- Depuis un nombre décimal : le nombre sera arrondi vers zéro.

27

Manipulation des types de données

Type entier

```
<?php
// un nombre décimal
$a = 1234;
// un nombre octal (équivalent à 83 en décimal)
$a = 0123;
// un nombre octal (à partir de PHP 8.1.0)
$a = 0o123;
// un nombre hexadécimal (équivalent à 26 en décimal)
$a = 0x1A;
// un nombre binaire (équivalent à 255 en décimal)
$a = 0b11111111;
// un nombre décimal (à partir de PHP 7.4.0)
$a = 1_234_567;
?>
```

28

Manipulation des types de données

Type nombre à virgules

Le type "nombre décimal" ou Float en anglais.

- Pour convertir explicitement une valeur en un float, utiliser le mot-clé (float).
- Depuis une chaîne de caractères : Si une chaîne est numérique alors elle sera transformée en sa valeur flottante correspondante, sinon elle sera convertie à zéro.
- Depuis un booléen : false correspond à 0.0, et true correspond à 1.0
- Depuis d'autres types : la conversion est effectuée d'abord en int puis en float

29

Manipulation des types de données

Type nombre à virgules

```
<?php
$a = 1.234;
$b = 1.2e3;
$c = 7E-10;
$d = 1_234.567; // à partir de PHP 7.4.0
$epsilon = 0.00001;
$f = 1.23456780;
?>
```

30

Manipulation des types de données

Type chaine de caractères



Information

La façon la plus simple de spécifier une chaîne de caractères est de l'entourer de guillemets simples (le caractère ').

- Pour spécifier un guillemet simple littéral, il doit être échappé à l'aide d'un antislash (\)
- Pour spécifier un antislash littéral, doublez-le (\\)
- Les séquences d'échappement pour les caractères spéciaux ne seront pas interprétées
- Un retour à la ligne dans la chaîne de caractère permet d'avoir une nouvelle ligne.

31

Manipulation des types de données

Type chaine de caractères



Information

Si la chaîne de caractères est entourée de guillemets doubles (le caractère ") :

- Les noms de variables seront interprétés.
- \n : Fin de ligne (LF ou 0x0A (10) en ASCII)
- \r : Retour à la ligne (CR ou 0x0D (13) en ASCII)
- \t : Tabulation horizontale (HT ou 0x09 (9) en ASCII)
- \v : Tabulation verticale (VT ou 0x0B (11) en ASCII)
- \e : échappement (ESC ou 0x1B (27) en ASCII)
- \f : Saut de page (FF ou 0x0C (12) en ASCII)
- \\ : Antislash
- \\$: Signe dollar
- \" : Guillemet double

32

Manipulation des types de données

Type chaîne de caractères

Exercice

Afficher : Ahmed a dit : "J'ai un autre exemple ici"

Afficher : Voulez-vous supprimer C:*.*?

Afficher : Ceci n'affichera pas \n de nouvelle ligne

- \$traitees= "égales à";
- \$ici = 2;

Afficher : Les variables ne seront pas \$traitees \$ici

Afficher : Les variables ne seront pas égales à 2

33

Manipulation des types de données

Type chaîne de caractères

Exercice

Afficher : Ahmed a dit : "J'ai un autre exemple ici"

Réponse : echo 'Ahmed a dit : "J\'ai un autre exemple ici";

Afficher : Voulez-vous supprimer C:*.*?

Réponse : echo 'Voulez-vous supprimer C:*.*?';

Afficher : Ceci n'affichera pas \n de nouvelle ligne

Réponse : echo 'Ceci n\'affichera pas \n de nouvelle ligne';

Afficher : Les variables ne seront pas \$traitees \$ici

Réponse : echo 'Les variables ne seront pas \$traitees \$ici';

Afficher : Les variables ne seront pas égales à 2

Réponse : echo "Les variables ne seront pas \$traitees \$ici";

34

Manipulation des types de données

Type tableau

Syntaxe d'un tableau est:

- **clé** => **valeur**
- La **clé** peut être soit un int, soit une chaîne de caractères.
- La **valeur** peut être de n'importe quel type.
- Structure de langage **array()**. La syntaxe courte remplace array() par **[]**
- Le tableau prend un nombre illimité de paramètres, chacun séparé par une virgule

35

Manipulation des types de données

Type tableau

```
<?php
// signifie la 0 => Salut, 1 => Aloha, 2=>Hello
$tableau1 = array("Salut", "Aloha", "Hello");
Echo $tableau1[0]; // Afficher : Salut
// signifie la 0 => Salut, 1 => Aloha, 2=>Hello
$tableau2 = ["Salut", "Aloha", "Hello"];
Echo $tableau2[0]; // Afficher : Salut

$tableau3 = array(
    "0" => "Salut",
    "1" => "Aloha",
    "2" => "Hello",
);
Echo $tableau3[2]; // Afficher : Hello
$tableau4 = [
    "0" => "Salut",
    "1" => "Aloha",
    "2" => "Hello",
];
Echo $tableau4[2]; // Afficher : Hello
?>
```

36

Manipulation des types de données

Type objet

Pour créer un nouvel objet, le mot clé `new` est utilisé afin d'instancier une classe.

Une classe définit le comportement d'un objet. La classe ne contient aucune donnée.

Un objet est une instance d'une classe qui contient des données.

Un membre est une variable qui appartient à un objet.

Une méthode est une fonction qui appartient à un objet et a accès à ses membres.

Un constructeur est une méthode spécifique qui est exécutée lorsqu'un objet est créé.

37

Manipulation des types de données

Type itérable

- Un itérable est un pseudo-type introduit en PHP 7.1. Il accepte n'importe quel tableau ou objet implémentant l'interface `Traversable`. (Interface permettant de détecter si une classe peut être parcourue en utilisant **`foreach`**).
- Tous les tableaux sont des itérables.
- Un tableau peut être utilisé comme argument d'une fonction qui nécessite un itérable.

38

Manipulation des types de données

Type itérable

Exemple

```
<?php
$week = ['Lundi','Mardi','Mercredi','Jeudi','Vendredi','Samedi','Dimanche'];

foreach($week as $key => $day){
echo "$day<br>";
}

?>
```

39

Manipulation des types de données

Modification de types

(int), (integer) :
modification en int

(bool), (boolean) :
modification en bool

(float), (double), (real) :
modification en float

(string) :
modification en string

(array) :
modification en array

(object) :
modification en object

40

Contrôles de flux et boucles

L'instruction if else

- Si l'expression vaut true, PHP exécutera l'instruction et si elle vaut false, l'instruction sera ignorée.
- Les instructions après le else ne sont exécutées que si l'expression du if est false.

```
if (expression)
    commandes
else
    commandes
```

41

Contrôles de flux et boucles

L'instruction elseif

```
if (expression)
    commandes
elseif(expression)
    commandes
elseif(expression)
    commandes
elseif(expression)
    commandes
else
    commandes
```

42

Contrôles de flux et boucles

Exercice

Déclarer une variable \$sexe qui contient une chaîne de caractères. Créer une condition qui affiche un message différent en fonction de la valeur de la variable.

43

Contrôles de flux et boucles

Réponse

```
<?php
$sexe = 'homme';
if($sexe == 'femme') :
    echo 'Bonjour Madame.';
elseif($sexe == 'homme') :
    echo 'Bonjour Monsieur.';
else :
    echo 'Bonjour sexe inconnu.';
endif;
?>
```

44

Contrôles de flux et boucles

L'instruction switch

- L'instruction switch équivaut à une série d'instructions if.
- Un cas spécial est default. Ce cas est utilisé lorsque tous les autres cas ont échoué.
- PHP continue d'exécuter les instructions jusqu'à la fin du bloc d'instructions du switch, ou bien dès qu'il trouve l'instruction **break**.
- Dans une commande switch, une condition n'est évaluée qu'une fois, et le résultat est comparé à chaque case.

45

Contrôles de flux et boucles

L'instruction switch

```
<?php
switch ($i) {
    case 0:
        echo "i égal 0";
        break;
    case 1:
        echo "i égal 1";
        break;
    case 2:
        echo "i égal 2";
        break;
    default:
        echo "i n'est ni égal à 2, ni à 1, ni à 0.";
}
?>
```

46

Contrôles de flux et boucles

Exercice:

Écrire un programme PHP qui affect le mois actuelle en format numérique a une variable et puis affiche le mois en format texte en utilisant **switch**.

Vous pouvez utilisé l'instruction `date("m")` pour récupérer le mois actuelle.

47

Contrôles de flux et boucles

Réponse

```
<?php
$numeroDeMois = date("m");
switch ($numeroDeMois)
{
    case 1 : $moisFrancaisActuel= 'Janvier' ; break;
    case 2 : $moisFrancaisActuel= 'Février'; break;
    case 3 : $moisFrancaisActuel= 'Mars'; break;
    case 4 : $moisFrancaisActuel= 'Avril'; break;
    case 5 : $moisFrancaisActuel= 'Mai'; break;
    case 6 : $moisFrancaisActuel= 'Juin'; break;
    case 7 : $moisFrancaisActuel= 'Juillet'; break;
    case 8 : $moisFrancaisActuel= 'Aout'; break;
    case 9 : $moisFrancaisActuel= 'Septembre'; break;
    case 10 : $moisFrancaisActuel= 'Octobre'; break;
    case 11 : $moisFrancaisActuel= 'Novembre'; break;
    case 12 : $moisFrancaisActuel= 'Décembre'; break;
}
echo "Nous sommes au mois de ".$moisFrancaisActuel;
?>
```

Contrôles de flux et boucles

L'instruction while

- PHP exécute l'instruction tant que l'expression de la boucle while est évaluée comme true. Si l'expression du while est false avant la première itération, l'instruction ne sera jamais exécutée.

```
while(expression) :
    commandes
endwhile;
```

L'instruction do-while

- La principale différence par rapport à la boucle while est que la première itération de la boucle do-while est toujours exécutée.

```
do
    commandes
while(expression);
```

49

Contrôles de flux et boucles

L'instruction for

- **expr1**: est évaluée (exécutée), au début de la boucle.
- **expr2**: est évaluée, au début de chaque itération. Si l'évaluation vaut true, la boucle continue et les commandes sont exécutées. Si l'évaluation vaut false, l'exécution de la boucle s'arrête.
- **expr3**: est évaluée(exécutée), à la fin de chaque itération.

Les expressions peuvent éventuellement être laissées vides ou peuvent contenir plusieurs expressions séparées par des virgules.

```
for (expr1; expr2; expr3)
    commandes
```

50

Contrôles de flux et boucles

L'instruction foreach

- foreach ne fonctionne que pour les tableaux et les objets.
- La forme suivante passe en revue le tableau `iterable_expression`. À chaque itération, la valeur de l'élément courant est assignée à `$value`.

```
foreach (iterable_expression as $value){
    //commandes
}
```

- La forme suivante assignera en plus la clé de l'élément courant à la variable `$key` à chaque itération.

```
foreach (iterable_expression as $key => $value){
    //commandes
}
```

51

Contrôles de flux et boucles

L'instruction break

- L'instruction `break` permet de sortir d'une structure `for`, `foreach`, `while`, `do-while` ou `switch`.
- `Break` accepte un argument numérique optionnel qui vous indique à combien de structures emboîtées doivent être interrompues.

```
<?php
$i = 0;
while (++$i) {
    switch ($i) {
        case 5:
            echo "At 5<br />\n";
            break 1; /* Termine uniquement le switch. */
        case 10:
            echo "At 10; quitting<br />\n";
            break 2; /* Termine le switch et la boucle while. */
        default:
            break;
    }
}
?>
```

52

Contrôles de flux et boucles

L'instruction continue

- L'instruction continue est utilisée dans une boucle pour contourner l'instruction de l'itération actuelle et poursuivre l'exécution sous la condition évaluée, en commençant l'itération suivante.
- Continue accepte un argument numérique optionnel qui vous indique à combien de structures emboîtées doivent être interrompues.
- La structure continue s'applique aux structures switch et se comporte de la même manière que break.

```
<?php
for ($i = 0; $i <= 10; ++$i) {
    if ($i % 2)
        continue;
    print "$i\n"; // Affiche : 0 2 4 6 8 10
}
?>
```

53

Contrôles de flux et boucles

Exercice

En utilisant la boucle for, écrire un script qui produit le résultat ci-dessous.

```
*
**
***
****
*****
*****
*****
```

54

Contrôles de flux et boucles

Réponse

```
<?php
for($i = 1;$i <= 6;$i++)
{
    for($j=1;$j<=$i;$j++)
    {
        echo '*';
    }
    echo '<br />';
}
?>
```

55

Manipulation des types de données

Exercice

Déclarer une variable avec le nom de votre choix et avec la valeur **0**. Tant que cette variable n'atteint pas **20**, il faut :

- l'afficher ;
- incrémenter sa valeur de **2** ;
- Si la valeur de la variable est égale à **10**, la mettre en valeur avec la balise HTML appropriée pour l'afficher en gras.

56

Manipulation des types de données

Réponse

```
<?php
$k = 0;
while($k <= 20) {
    if($k == 10) {
        echo '<strong>'.$k.'</strong>';
    }
    else
    { echo $k;
    }
    echo '<br />';
    $k = $k + 2; }
?>
```

57

Formulaires simples

Formulaires HTML

- Lorsqu'un formulaire est envoyé à un script PHP, toutes les variables du formulaire seront automatiquement disponibles dans le script.

Exemple:

Ecrire un formulaire qui demande de remplir le nom et prénom avec un bouton OK. Lorsque un visiteur remplit le formulaire, et clique sur le bouton OK, le fichier action.php est appelé:

58

Formulaires simples

Formulaires HTML

Exemple:

Ecrire un formulaire qui demande de remplir le nom et prénom avec un bouton OK. Lorsque un visiteur remplit le formulaire, et clique sur le bouton OK, le fichier `action.php` est appelé:

```
<form action="action.php" method="post">
<p>Votre nom : <input type="text" name="nom" /></p>
<p>Votre prénom : <input type="text" name="prenom" /></p>
<p><input type="submit" value="OK"></p>
</form>
```

59

Transmission de variables (GET, POST)

`$_GET`

- Elle donne les valeurs des informations indiquées dans l'url.
- Les informations après le point d'interrogation ? d'une URL sont en réalité des données que l'on fait transiter d'une page à une autre.

Exemple:

- Le site s'appelle : `lesite.com`
- Ma page PHP de utilisateur : `action.php`
- Pour accéder à la page de l'utilisateur, l'URL sera : `http://www.lesite.com/action.php`
- Pour envoyer les informations de l'utilisateur à la page php nous utiliserons :
- `http://www.lesite.com/action.php?nom=Rasmus&prenom=Lerdorf`
- Les paramètres sont séparés par le symbole `&`

60

Transmission de variables (GET, POST)

\$_GET (Exemple de formulaire):

```
<form action="action.php" method="get">
<p>Votre nom : <input type="text" name="nom" /></p>
<p>Votre prénom : <input type="text" name="prenom" /></p>
<p><input type="submit" value="OK"></p>
</form>
```

- Quand l'utilisateur clique sur le bouton **OK**, l'URL envoyé au serveur sera visible par le visiteur dans la barre d'adresse comme ceci :
http://www.lesite.com/action.php?nom=Rasmus&prenom=Lerdorf
- Le fichier **action.php** peut maintenant employer la variable superglobale **\$_GET** pour récupérer les données du formulaire.

```
<?php
Echo "Bonjour $_GET["nom"] $_GET["prenom"] et bienvenue";
?>
```

61

Transmission de variables (GET, POST)

\$_POST et \$_REQUEST

- PHP **\$_POST** est une variable superglobale PHP qui est utilisée pour collecter des données de formulaire après avoir soumis un formulaire HTML avec **method="post"**.
- **\$_POST** est également utilisé pour passer des variables.
- Lorsque nous envoyons des variables via un formulaire, la variable est récupérée de cette façon **\$_POST['Variable']** et non plus **\$Variable**.

62

Transmission de variables (GET, POST)

\$_POST et \$_REQUEST

```
<form action="action.php" method="post">
<p>Votre nom : <input type="text" name="nom" /></p>
<p>Votre prénom : <input type="text" name="prenom" /></p>
<p><input type="submit" value="OK"></p>
</form>
```

- Pour accéder aux variables du formulaire

```
<?php
echo $_POST['nom'];
echo $_REQUEST['prenom'];
?>
```

63

Variables d'environnement

\$_FILES

- Variable de téléchargement de fichier via HTTP:
 - `$_FILES['nom_de_la_variable']['name']` : Le nom original du fichier qui provient de la machine du client
 - `$_FILES['nom_de_la_variable']['type']` : Le type MIME du fichier
 - `$_FILES['nom_de_la_variable']['size']` : La taille du fichier en bytes (soit 8 bits ou un octet)
 - `$_FILES['nom_de_la_variable']['tmp_name']` : Le nom temporaire du fichier stocké sur le serveur
 - `$_FILES['nom_de_la_variable']['error']` : Le code erreur associé à l'upload

\$_SESSION

- Variables de session. Liste des fonctions dans <https://www.php.net/manual/fr/ref.session.php>

\$_ENV

- Variables d'environnement

\$_COOKIE

- -Cookies HTTP

64

Exercice

1. Créez un formulaire info.htm comportant 2 champs de texte nom, prénom, un menu d'options sexe (M ou F).
2. Vous ferez appel à un script info.php pour traiter les données du formulaire. Elles seront envoyées par la méthode GET.